A Register-Graph Presentation of iPRA

Removing primitive successor and initialization with an explicit whole-domain bi-interpretation

Research report on the ConciseIPRA proposal

November 10, 2026

Abstract

We replace the numerical syntax $0, S, I(n, a; u.h)$ of the current *ConciseIPRA* proposal by $0, J(n; i, r.h)$. The register of J always starts at one. A zero instruction increments it; an instruction $S(t)$ loads t. Successor is the abbreviation $S(a) = J(a; i, r.0)$, and no independent initializer is primitive. A transition-graph biconditional also supplies numerical coverage. Predecessor and an unrestricted selector have short derivations without induction. Every primitive recursion then compiles by one extra iteration, again without induction or pairing. Literal-equation induction recovers quantifier-free induction, and both interpretation composites are internally equal to the identity on the entire numerical domain.

This is a mathematical simplification of the term grammar and arithmetic bootstrap. It is *not* an established improvement on the smallest complete proof-checker size: the transition axiom is richer, and the counter-aware body has two bound variables rather than one. We supply a readable complete checker for the new theory and reproducible measurements. A separate 288-AST-node implementation of the inherited scalar calculus is included as an ancillary implementation result; it does not implement the new arithmetic. No absolute simplest axiomatization or proof-assistant verification is claimed.

Main construction.

$$t ::= x \mid 0 \mid J(n; i, r.h), \qquad S(a) := J(a; i, r.0).$$

The initializer is recovered by the explicit compiler

$$R(n, a; u, v.h) = J\big(Sn; i, r.S(C(i, a, h(Pi, r)))\big).$$

Its equations are $R(0, a; h) = a$ and $R(Sn, a; h) = h(n, R(n, a; h))$.

Reading guide. The complete theory is in Section 2; the direct recursion compiler is in Section 4; equality reflection and the two internal round trips are in Sections 5–6. Section 7 separates the mathematical result from the executable-checker measurements.

1 Result and comparison boundary

The comparison is pinned to ProveIt commit `8662bc0a00f4e77e482f7d141d7a4f919f70f9aa`, read on November 10, 2026 in the user's time zone. Its `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/README.md` describes a native scalar calculus with terms $0, S, I$, formulas built from equality and two binders, and an ordinary realization T_{eq}. The ordinary realization has intuitionistic equality logic, scalar iterator equations, numerical coverage, $0 \neq S0$, and contextual literal-equation induction [1, 2].

The new arithmetic uses ordinary first-order intuitionistic logic. The logical rules are specified in Section 2.2; they are part of the presentation and are included in the executable checker. We do not compare an arithmetic-only program against a complete old checker.

Theorem 1.1 (Main result)**.** *The theory T_J defined in Section 2 is bi-interpretable with full first-order intuitionistic primitive recursive arithmetic with quantifier-free induction. Both domains are the whole numerical domain, interpreted equality is actual equality, and the graph of each composite isomorphism is $x = y$. Consequently the two theories are equiconsistent.*

Here "full first-order" concerns the available formulas and logical rules. Induction is restricted to quantifier-free formulas in iPRA, and initially to literal equations in T_J. No induction on arbitrary quantified formulas is smuggled into the argument.

Feature	Current scalar arithmetic	Proposed register arithmetic
Primitive numerical constructors	$0, S, I$	$0, J$
Independent initializer	Argument of I	Derived by one extra step
Local variables in a body	One: previous state	Two: old counter and state
Predecessor bootstrap	A scalar iteration construction	Four short abbreviations
General primitive recursion	Derived using pairing	Direct one-line compiler
Computation specification	Two equations	One base equation and a graph biconditional
Numerical coverage	Separate in T_{eq}	Derived from the graph
Complete-checker byte record	529 normalized bytes	Not improved here

The counter variable is a real cost, as is the richer graph axiom. A one-body-variable version is given in Appendix A. That version retains the old binding arity while still eliminating primitive successor and the initializer, but its simulation of general primitive recursion passes through the scalar bridge.

2 Complete theory

2.1 Numerical syntax and finite descriptions

Terms are finite expressions

$$t ::= x \mid 0 \mid J(n; i, r.h).$$

The distinct variables i, r are bound *only in* h; they are not bound in n. Other variables in h are passive numerical parameters. Substitution is capture-free. We use the abbreviations

$$S(a) := J(a; i, r.0), \qquad 1 := S0, \qquad f(n, a) := J(n; i, r.a),$$

where the bound variables are chosen fresh for a. Fixed superscripts S^k, P^k mean finite composition, never exponentiation of an unspecified function.

The binding notation is a finite-description notation for a first-order language, not an additional function sort. More formally, start with the fixed zero-body description J_0. For every fixed finite body $h(i, r, \vec{p})$ built from earlier descriptions, introduce the first-order function symbol $J_h(n, \vec{p})$ and its displayed schemas. The symbol S is always the abbreviation J_0. Nested bodies are handled in the same finite dependency order. The zero-body schemas themselves use $S = J_0$; their standard interpretation is established in Section 6. No variable ranges over descriptions and no uniform evaluator is postulated inside the theory.

2.2 Logical language and rules

Formulas are generated by

$$A ::= (s = t) \mid \bot \mid (A \wedge A) \mid (A \vee A) \mid (A \to A) \mid \forall x A \mid \exists x A.$$

Negation abbreviates $A \to \bot$, and $A \leftrightarrow B$ abbreviates $(A \to B) \wedge (B \to A)$. Contexts Γ are finite reusable collections of assumptions. The logical inference rules are the following ordinary natural-deduction rules. Displayed auxiliary assumptions are discharged.

Rule	Premises	Conclusion
Hypothesis	$A \in \Gamma$	$\Gamma \vdash A$
Equality reflexivity	none	$\Gamma \vdash t = t$
Equality transport	$\Gamma \vdash s = t$, $\Gamma \vdash F(s)$	$\Gamma \vdash F(t)$
Falsity elimination	$\Gamma \vdash \bot$	$\Gamma \vdash A$
$\wedge$ introduction	$\Gamma \vdash A$, $\Gamma \vdash B$	$\Gamma \vdash A \wedge B$
$\wedge$ elimination	$\Gamma \vdash A \wedge B$	Either conjunct
$\vee$ introduction	$\Gamma \vdash A$ or $\Gamma \vdash B$	$\Gamma \vdash A \vee B$
$\vee$ elimination	$\Gamma \vdash A \vee B$, $\Gamma, A \vdash F$, $\Gamma, B \vdash F$	$\Gamma \vdash F$
$\to$ introduction	$\Gamma, A \vdash B$	$\Gamma \vdash A \to B$
$\to$ elimination	$\Gamma \vdash A \to B$, $\Gamma \vdash A$	$\Gamma \vdash B$
$\forall$ introduction	$\Gamma \vdash A(z)$, z fresh	$\Gamma \vdash \forall x A(x)$
$\forall$ elimination	$\Gamma \vdash \forall x A(x)$	$\Gamma \vdash A(t)$
$\exists$ introduction	$\Gamma \vdash A(t)$	$\Gamma \vdash \exists x A(x)$
$\exists$ elimination	$\Gamma \vdash \exists x A(x)$, $\Gamma, A(z) \vdash F$, z fresh	$\Gamma \vdash F$

For universal introduction, z is absent from Γ and the fixed parameters of A. For existential elimination, it is absent from Γ, F and the fixed parameters of the matrix. The template $F(v)$ in equality transport is any formula. Weakening, exchange, contraction, renaming, and assumption cut are admissible. There is no classical rule or excluded-middle axiom. These rules, together with the arithmetic schemas below, are the complete presentation.

2.3 Arithmetic schemas

For each fixed body $h(i, r, \vec{p})$, write $j_n = J(n; i, r.h)$ and $e_n = h(n, j_n, \vec{p})$. Use universal closure over the free numerical variables in the following schemas:

$$J(0; i, r.h) = 1, \qquad \text{(J0)} \tag{1}$$

$$J(Sn; i, r.h) = b \;\leftrightarrow\; \big((e_n = 0 \wedge b = Sj_n) \vee e_n = Sb\big), \qquad \text{(STEP)} \tag{2}$$

$$0 = 1 \;\to \bot. \qquad \text{(Sep)} \tag{3}$$

In STEP, b is an output variable distinct from the body's passive parameters before universal instantiation. All instances obtained by capture-free substitution are allowed.

The one arithmetic inference rule is contextual equation induction:

$$\frac{\Gamma \vdash s(0,\vec{p}) = t(0,\vec{p}) \qquad \Gamma, s(z,\vec{p}) = t(z,\vec{p}) \vdash s(Sz,\vec{p}) = t(Sz,\vec{p})}{\Gamma \vdash s(a,\vec{p}) = t(a,\vec{p})}. \tag{EI}$$

The variable z is fresh for Γ, the target a, and the fixed parameters. The invariant must be a *literal equation*; an equation under a quantifier is not eligible. Contexts may contain arbitrary first-order formulas. No further arithmetic axiom is primitive.

2.4 Operational meaning and intrinsic coverage

In the standard interpretation, $J(n;h)$ executes n instructions with an old-counter variable i and a register r, initially $r = 1$. The fixed finite body returns an instruction value. Zero increments the register; a value $S(t)$ loads t. Thus, for each fixed body, this is ordinary primitive recursion.

The right-to-left direction of STEP immediately gives the two useful conditional computation rules

$$e_n = 0 \implies J(Sn;h) = Sj_n, \tag{4}$$
$$e_n = St \implies J(Sn;h) = t. \tag{5}$$

Their premises are proved equations, not tests performed by an untrusted evaluator.

Lemma 2.1 (Functional coverage)**.** *Define $W(a) := f(1,a)$. Before any induction,*

$$\mathsf{T}_J \vdash a = 0 \vee a = S(W(a)).$$

In particular, $\forall a(a = 0 \vee \exists x\, a = Sx)$ is derivable.

Proof. Take the constant body $h(i,r) = a$, set $n = 0$ and $b = f(1,a)$ in STEP, and use reflexivity on its left side. Its right side is $(a = 0 \wedge b = S1) \vee a = Sb$. Forget the unused conjunct. □

Conversely, J0, the two conditional rules, and numerical coverage prove STEP. For its left-to-right direction, split coverage for e_n. At zero use the increment equation; at St use the load equation and the given equality of outputs. The right-to-left direction is computation. This equivalence does not require induction, separation, or successor injectivity. The biconditional is essential: its left-to-right direction alone would not assert deterministic computation.

One can package J0 and STEP into a single graph schema by explicitly distinguishing a zero counter from a successor counter. Such packaging also works for the old iterator, so a smaller number of named schemas alone is not the claimed improvement.

3 A short arithmetic bootstrap

All endpoint equations in this section use only equality and the right-to-left direction of STEP together with J0. They need neither induction nor coverage. Consequently none of the constructions relies on already having arithmetic comparison or an equality characteristic.

3.1 Constant instructions and predecessor

The abbreviation $f(n,a) = J(n;i,r.a)$ satisfies

$$f(0,a) = 1, \qquad f(n,0) = Sn, \qquad f(Sn,Sa) = a. \tag{6}$$

The middle equation is the definition of S. The last is a load step whose premise is reflexivity $Sa = Sa$.

Define

$$Z(x) = f(x, 1), \quad W(x) = f(1, x), \quad D(x) = f(x, Sx), \quad P(x) = W(D(x)).$$

Then direct instances of (6) give

$$Z(0) = 1, \qquad Z(Sx) = 0, \tag{7}$$
$$W(0) = S1, \qquad W(Sx) = x, \tag{8}$$
$$D(0) = 1, \qquad D(Sx) = Sx, \tag{9}$$
$$P(0) = 0, \qquad P(Sx) = x. \tag{10}$$

For example, $P(0) = W(1) = 0$ and $P(Sx) = W(Sx) = x$. Thus P is a predecessor throughout the internal domain, and successor injectivity follows by applying P to $Sx = Sy$.

Using Sep, successor is never zero: an equality $Sx = 0$ implies $Z(Sx) = Z0$, hence $0 = 1$, and therefore $\bot$. These facts are consequences, not successor axioms silently appended to the presentation.

3.2 Unrestricted selection

Put

$$G(c, a) = P^2 f(Sa, Zc).$$

Because $Z(0) = 1$ and $Z(Sn) = 0$, equations (6) and (10) give

$$G(0, a) = 0, \qquad G(Sn, a) = a. \tag{11}$$

Indeed, the inner f is zero in the first case and S^2a in the second case. Now define

$$C(c, a, b) := P^2 f\big(Sa,\, G(c, S^3 b)\big). \tag{12}$$

At $c = 0$, the instruction $G(c, S^3b)$ is zero, so the inner f is S^2a. At $c = Sn$, the instruction is S^3b, and the positive counter Sa loads S^2b. Hence

$$\boxed{C(0, a, b) = a, \qquad C(Sn, a, b) = b.} \tag{13}$$

This is a selector of arbitrary numerical outputs. It is not limited to Boolean results or to values bounded by a preselected initializer. The direct use of the old register in the increment branch is what escapes the single-body information restriction of tied-seed iterations discussed in the current research packages [5].

4 Every primitive recursion in one extra step

Consider any fixed intended recursion with passive parameters:

$$F(0, \vec{p}) = a(\vec{p}), \qquad F(Sn, \vec{p}) = h(n, F(n, \vec{p}), \vec{p}).$$

Assume a, h have already been translated to J terms. Choose fresh local variables i, r and define

$$H(i, r, \vec{p}) := S\big(C(i, a(\vec{p}), h(Pi, r, \vec{p}))\big), \tag{14}$$
$$R(n, a; u, v.h) := J(Sn; i, r.H(i, r, \vec{p})). \tag{15}$$

The old counter distinguishes the first update from the subsequent ones. The instruction is always a successor, so every update is a load.

Theorem 4.1 (Direct recursion compiler)**.** *Without induction, coverage, pairing, or equality reflection,*

$$R(0,a;h)=a, \qquad R(Sn,a;h)=h(n,R(n,a;h)).$$

Proof. The first update has counter $i=0$ and old register 1. By (13), its instruction is Sa, and one load gives $J(1;H)=a$.

For the next equation, put $r=J(Sn;H)=R(n,a;h)$. Its old counter is Sn, so (13) and $P(Sn)=n$ give

$$H(Sn,r)=S\big(h(n,r)\big).$$

One load therefore gives $J(S(Sn);H)=h(n,J(Sn;H))$, exactly the required equation. □

Initial functions and composition translate by literal term substitution; Theorem 4.1 handles each primitive-recursion node. External induction on the finite description constructs a J term for every primitive-recursive description and proves both of its defining equations internally. Nothing in this compiler quantifies over a class of functions inside the theory.

For example, define

$$x+y=R(y,x;u,v.Sv), \qquad d(x,y)=R(y,x;u,v.Pv), \qquad x\cdot y=R(y,0;u,v.v+x).$$

Their ordinary recursion equations follow immediately. These are finite J terms after expansion. A compact source grammar does not guarantee small expanded terms; the archive's actual syntax compiler makes that separate cost inspectable.

5 From equation induction to quantifier-free induction

This section adapts the repository's residual-reflection proof [3], using the already available Z,W,P,C. The proof is included to keep the main equivalence independent of a hidden equality-test assumption. All internal inductions below have a single literal equation as invariant.

5.1 Elementary identities

The recursion equations for addition and d are

$$x+0=x, \qquad x+Sy=S(x+y), \tag{16}$$

$$d(x,0)=x, \qquad d(x,Sy)=P(d(x,y)). \tag{17}$$

Equation induction yields

$$0+y=y, \qquad (Sx)+y=S(x+y), \tag{18}$$

$$d(Sx,Sy)=d(x,y), \qquad d(Sa,a)=1. \tag{19}$$

For the first two identities, induct on y using (16). For cancellation, the zero case is $P(Sx)=x$; the successor step applies P to the induction equation. For the diagonal, the zero case is $d(1,0)=1$ and the successor case is cancellation. In particular, $(Sy)+r=y+Sr$ follows from (18).

Coverage, the constructor laws, and (7) prove

$$Z(v)=0\vee Z(v)=1, \qquad Z(v)=1\leftrightarrow v=0.$$

Likewise (8) proves

$$W(v)=0\leftrightarrow v=1. \tag{20}$$

For the forward implication in (20), split coverage of v. Its zero branch gives $W0=2\neq 0$, a contradiction; its successor branch $v=Sx$ gives $x=0$. No induction is involved.

5.2 One residual equation reflects actual equality

Lemma 5.1. *For every x, y,*

$$C\big(d(x,y), x, y + d(x,y)\big) = x. \tag{21}$$

Proof. Hold x fixed and induct on y. At zero the equation is $C(x, x, 0 + x) = x$; both branches equal x, by (18) and coverage.

For the step write $r = d(x,y)$ and $r' = Pr = d(x, Sy)$. If $r' = 0$, the new selector directly returns x. Otherwise coverage writes r' as a successor. Coverage of r cannot choose zero, since that would make $r' = P0 = 0$; in its successor branch $r = S(Pr) = Sr'$. The induction equation now reduces to $y + r = x$. Meanwhile

$$(Sy) + r' = y + Sr' = y + r = x.$$

This is the branch selected by the successor control r' in the new equation. Every split is supplied by coverage; the induction invariant itself remains (21). □

Applying the lemma at $x = Sa, y = b$ shows

$$d(Sa, b) = 1 \leftrightarrow a = b. \tag{22}$$

In the forward direction it gives $b + 1 = Sa$, hence $Sb = Sa$ and then $b = a$ by P. The converse is the diagonal identity. Thus the finite term

$$e(a,b) := Z\big(W(d(Sa,b))\big) \tag{23}$$

has the internally proved properties

$$e(a,b) = 0 \vee e(a,b) = 1, \qquad e(a,b) = 1 \leftrightarrow a = b. \tag{24}$$

It tests actual equality on the whole domain, not equality only between standard numerals or a separately postulated congruence.

5.3 Boolean compilation of formulas

For each fixed quantifier-free formula A, define its characteristic term by recursion on its finite syntax:

$$\begin{aligned} \chi_{s=t} &= e(s,t), & \chi_{\bot} &= 0, \\ \chi_{A\wedge B} &= C(\chi_A, 0, \chi_B), & \chi_{A\vee B} &= C(\chi_A, \chi_B, 1), \\ \chi_{A\to B} &= C(\chi_A, 1, \chi_B). \end{aligned}$$

An external structural induction proves inside T_J

$$(\chi_A = 0 \vee \chi_A = 1) \quad \wedge \quad (\chi_A = 1 \leftrightarrow A). \tag{25}$$

For atoms use (24); for $\bot$ use Sep and explosion. For conjunction, the zero antecedent branch returns zero and excludes A, while the one branch returns χ_B. For disjunction, those branches return χ_B and one, respectively. For implication, the zero branch proves $\neg A$ and returns one, while the one branch already proves A and returns χ_B. These are constructive case arguments using the proved two-value range, not excluded middle for arbitrary formulas.

Given a quantifier-free base $A(0)$ and a contextual induction step $A(z) \vdash A(Sz)$, use (25) to obtain the corresponding base and step for the literal equation $\chi_A = 1$. EI gives that equation at any target, and (25) recovers A there. The retained context is unchanged, including any quantified assumptions. This proves full contextual quantifier-free induction.

6 The interpretations and both internal round trips

6.1 Interpreting T_J in iPRA

Translate each fixed body recursively to primitive-recursive syntax. Using the standard predecessor and selector of iPRA, define

$$J_h(0,\vec{p}) = 1,$$

$$J_h(Sn,\vec{p}) = C\big(h(n, J_h(n,\vec{p}),\vec{p}),\ S(J_h(n,\vec{p})),\ P(h(n, J_h(n,\vec{p}),\vec{p}))\big). \qquad \text{(J in iPRA)}$$

This is a valid fixed primitive-recursive description. For the zero body, equation induction proves $J_0(n) = S_{\mathsf{iPRA}}n$: the base is one and the step increments. Therefore the new abbreviation S is interpreted as the old successor, provably at every internal number, not just in the standard model.

J0 and Sep now translate to theorems. To prove STEP, use numerical coverage for the instruction. At zero the update increments, and at a successor it loads its predecessor. Both directions of the biconditional follow by equality reasoning. EI translates to a quantifier-free induction instance. Ordinary logic and equality are preserved literally.

6.2 Interpreting iPRA in T_J

Translate zero and variables identically and translate successor by J_0. Translate initial functions and composition directly, and each primitive-recursion node using Theorem 4.1. Its defining equations hold internally. Successor injectivity and separation were proved from P and Z; Section 5 supplies every quantifier-free induction instance. All logical connectives, quantifiers, equality, and falsity are unchanged. This validates every axiom and inference rule of full first-order iPRA in the target.

6.3 Why these are bi-interpretations

Mutual interpretability alone would be insufficient. We prove the composites internally.

For an ordinary primitive-recursive description, use external induction on its finite construction. Its double translation has the same initial values and successor equation as the original, after using the already proved equalities for earlier descriptions. Literal-equation induction comparing the two outputs proves their equality at every counter, with passive parameters fixed.

For a J description, the double translation has initial value one and the same increment/load transition. Induct on the single equation equating its output with the original J output. In the step, the induction equation and the already established translation equalities for the body identify the two instruction values. Split coverage for that common value. The zero branch increments equal old registers; the successor branch loads the same number. The output equation follows in both branches. This proof uses no induction on a formula with a quantifier.

The domains in both interpretations are defined by $x = x$. Interpreted equality is $x = y$. The proved operation equalities show that the graph $x = y$ is an internal total bijection preserving every fixed symbol in both composites. Structural induction on formulas then gives the formula round trips, since the logical operations were left unchanged. This proves Theorem 1.1. Translations preserve $\bot$ and transform finite proofs effectively, so the equiconsistency claim follows as well.

7 Executable artifacts and fair complexity accounting

The archive distinguishes three kinds of code.

1. `code/checker.py` checks proofs in the new counter-aware T_J theory, including logical rules, scope conditions, substitution, arithmetic schemas, equation induction, and the public JSON interface. Its tests include valid proofs and invalid scope and induction attempts.

2. The semantic evaluators and finite-description compiler execute the displayed constructions on standard natural numbers. They are useful implementation checks; they are not proofs of the universal metatheorems or proof checkers.

3. `ancillary/checker_288.py` is a smaller AST profile of the *old* native scalar calculus. It does not validate J proofs. Its parent source and test certificates are frozen in the archive.

All source measurements count the entire standalone module after CPython 3.12.14 `ast.unparse` normalization plus one final newline. Tokens exclude encoding, end-marker, layout, and comment tokens; AST nodes are counted by `ast.walk`. Gzip uses level nine and a zero timestamp, and is informational rather than an optimization target. The exact machine-readable measurements are supplied in the archive; the following table reports the relevant complete checkers.

Complete standalone program	Bytes	Tokens	AST	Gzip-9
Old byte leader	529	266	310	333
Old AST leader	613	263	289	349
Direct parent of ancillary profile	576	261	292	345
New ancillary old-calculus profile	604	261	288	355
New counter-aware J reference checker	13633	2814	3060	3564

The ancillary profile replaces a recursive continuation by a loop and uses immutable tuple contexts. It preserves the parent's prevalidated elimination protocol and global fresh-name order. It reduces the inspected AST minimum by one node and ties the inspected token minimum; it is larger in bytes and gzip size. This is an implementation result with inherited mathematics, not evidence that the new register theory has a smaller checker.

The generic iterable opener in that ancillary profile inherits a malformed-input qualification: in an ideal unlimited interpreter, certain nonempty strings can cause unbounded traversal. The soundness claim concerns finite successful executions, and completeness uses finite canonical array/integer certificates. The new readable J checker validates its concrete syntax explicitly. Neither program assumes that a fixed physical computer checks arbitrarily large proofs.

The written metatheorems have been independently reviewed in this research run but have not been formalized in Lean or Rocq. Finite tests are not substitutes for those metatheorems. The delivered comparison does not establish the requested absolute minimum, nor a new complete checker below the old 529-byte benchmark. The positive result is the smaller arithmetic grammar and the direct, short compiler, with the remaining implementation cost made explicit.

8 Further research questions

1. Can a native Q/D checker share its already verified equality major premise with increment/load computation so that the new grammar also reduces the complete trusted source? All conditional-premise, scope, and syntax checks must be charged.

2. Is there a substantially shorter numerical selector for the register basis? The current C has a short proof but expands to a large finite term. Source brevity, term size, and certificate size are different objectives.

3. Does the counter-aware presentation, or the one-variable scalar variant in Appendix A, yield the smaller complete checker after comparable implementations? The present results do not decide this tradeoff.

4. Can the existing Lean development be reused by proving the recursion compiler and both round trips as translations of finite descriptions? A formalization should preserve the distinction between fixed syntactic bodies and arbitrary functions in the metatheory.

5. Which parts of the graph specification can be weakened while retaining actual equality and whole-domain formula round trips? One-sided operational membership and standard-model agreement alone are insufficient.

A A single-body-variable variant

If preserving the old binding arity is preferred, use

$$t ::= x \mid 0 \mid J_s(n; r.h).$$

The register still starts at one, with the same increment/load instructions. Remove the old counter argument from the instruction in STEP. Define $S(a) = J_s(a; r.0)$. The f, Z, W, D, P, G, C bootstrap and intrinsic-coverage argument are unchanged because all their bodies are constant.

For an arbitrary initialized scalar iteration with body $h(r)$, put

$$\begin{aligned} H(v) &= S^3\big(C(Pv, a, h(P^2v))\big), \\ K(n) &= J_s(Sn; v.H(v)), \\ I(n, a; r.h) &= P^2K(n). \end{aligned}$$

One load computation gives, at every counter,

$$J_s(Sn; H) = S^2\big(C(P(J_s(n; H)), a, h(P^2J_s(n; H)))\big).$$

Every post-initial state is therefore visibly a double successor. At the first step, the previous register is one and $P1 = 0$, so $K0 = S^2a$. At the next step $PK(n)$ is a successor, and the selector chooses the update branch. Thus

$$K(Sn) = S^2h(P^2K(n)),$$

and applying P twice gives the two scalar-iterator equations. This compiler needs no induction or coverage: the displayed shape of the state is a direct computation consequence, not an inferred invariant.

Conversely, using the old scalar iterator, define

$$J_s(n; h) = I(n, 1; v.C(h(v), Sv, P(h(v)))).$$

Coverage validates its graph. Equation induction proves that its successor abbreviation agrees with the old successor. Recurrence uniqueness proves both composites internally, as in the main text. Hence this scalar alternative is bi-interpretable with T_{eq} and therefore with iPRA, using the repository's ordinary scalar bridge [4]. Its predecessor, selector, and initialization proofs are explicitly supplied here; the existing scalar-to-general-PR compiler is needed only for this optional variant, not for the main counter-aware theorem.

B Compatibility with the inherited native logical core

The new arithmetic can also be combined with the repository's native choice-D rules rather than ordinary logical notation. Conditional computation needs actual proof premises. Even though J0 no longer returns an arbitrary initializer, numerical reflexivity can still be derived without adding a primitive reflexivity rule:

1. J0 for the zero body is literally $1 = 1$.
2. A load step with constant instruction $1 = S0$ yields $J(1; 1) = 0$. Transporting that equation along itself yields $0 = 0$.
3. Increment with the zero body and any counter t yields the literal equation $S(St) = S(St)$, since $J(t; 0) = St$ by definition.
4. A load of the constant instruction $S(St)$ yields an equation with right side St; self-transport yields $St = St$.
5. A load of the constant instruction St yields an equation with right side t; self-transport yields $t = t$.

The derived selector then supports the inherited logical macros. This shows that the new basis is compatible with that equality bootstrap. It does not claim an unimplemented native J checker of the old checker's size. The executable J checker delivered here uses the explicit ordinary logical rules of Section 2.2.

References

[1] Vladimir Reshetnikov, *ProveIt: Concise presentations of intuitionistic primitive recursive arithmetic*, README, pinned commit `8662bc0a00f4e77e482f7d141d7a4f919f70f9aa`. Pinned README.

[2] ProveIt, *Complementary introductions with one local assumption*, `Research/certified_zero/reference/inherited_choice_basis.tex`, at the same pinned commit. Pinned source.

[3] ProveIt, *Equality reflection and quantifier-free induction*, `Research/certified_zero/reference/equality_reflection.tex`, at the same pinned commit. The residual argument in Section 5 is an attributed adaptation. Pinned source.

[4] ProveIt, *Arithmetic strength and both ordinary interpretation composites; pairing and primitive recursion*, `Research/certified_zero/reference/inherited_ordinary_bridge.tex` and `primitive_recursion.tex`, at the same pinned commit. Pinned source directory.

[5] ProveIt, *Local scopes and relational obstructions*, `Research/local_scopes/theory/relational_obstruction.tex`, at the same pinned commit. Pinned source.

[6] ProveIt, the complete checkers in `Research/neutral_scope`, `Research/coverage_and_kernel`, and `Research/prevalidated_elimination`, at the same pinned commit. Exact source paths, hashes, and reproductions are supplied with the archive.